Ciências ULisboa
Faculdade de Ciências da Universidade de Lisboa

Cloud Computing

Final Project Report: CloudAPIService

Group 11

Denis Ungureanu	56307
Frederico Prazeres	56269
Leonardo Monteiro	58250
Gustavo Henriques	64361

Prof. Mário Calha

MSc in Computer Science and Engineering

Junho 2025

1 Motivation, Dataset Characterization, Business Capabilities, and Use Cases

1.1 Motivation

This project was designed to simulate key features of platforms like LinkedIn by exposing job postings, company reviews, and employee data through a structured API. The goal was to help users extract meaningful insights—like salary trends or remote job availability—while applying cloud-native technologies to build a scalable and secure backend.

1.2 Dataset Characterization

Source Data

- **employee_counts.csv**: *Auxiliary table with number of employees and companyId (1.04 MB)*
- **job_postings.csv**: *Table with LinkedIn job postings (516.84 MB)*
- **job_reviews.csv**: *Table with Glassdoor Job Reviews (292.85 MB)*

Data Relationships

All three datasets are connected through a shared `companyId` field, enabling cross-referencing between job postings, reviews, and company size data.

Data Sensitivity

Default encryption provided by GKE for Persistent Disks (PostgreSQL data) and Cloud Storage.

Business Capabilities

1. Obtain statistics about the job market;
2. Perform advanced job searches similar to LinkedIn;
3. Analyze reviews about jobs and companies;

Use Cases

- Search for remote jobs based on a specific city, keyword, or company.
- Retrieve the average salary for a given job title.
- Find the best-rated companies based on attributes like work-life balance, culture, etc.
- Identify the city with the highest ratings for specific attributes (e.g., work-life balance, culture).
- Suggest the best companies to apply to based on a candidate's profile (skills, location, preferences).
- Search for jobs by title and city, returning the average rating for that position in the specified city.
- Search for jobs filtered by the top-rated companies.

2 API Specification

2.1 api-interface Service Endpoints

Endpoint: /jobs/search/remote

- **Path:** */jobs/search/remote*
- **HTTP Method:** *GET*
- **Request Parameters:** *city, keyword, company*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
[
  {
    "id": "123",
    "title": "Software Engineer",
    "companyName": "TechCorp",
    "description": "Remote job opportunity...",
    "location": "Remote",
    "views": 234
  }
]
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/search/average-salary

- **Path:** */jobs/search/average-salary*
- **HTTP Method:** *GET*
- **Request Parameters:** *title*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
{
  "title": "Marketing Coordinator",
  "average_salary": 55000,
  "currency": "USD"
}
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/search/best-companies

- **Path:** */jobs/search/best-companies*
- **HTTP Method:** *GET*
- **Request Parameters:** *None*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

  ```
  [ { "id": "c1", "name": "TechCorp" } ]
  ```

- **Error Response Format(s):** *500*

Endpoint: /jobs/search/best-cities

- **Path:** */jobs/search/best-cities*
- **HTTP Method:** *GET*
- **Request Parameters:** *None*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

  ```
  [ { "name": "San Francisco" } ]
  ```

- **Error Response Format(s):** *500*

Endpoint: /jobs/search/jobs-with-rating

- **Path:** */jobs/search/jobs-with-rating*
- **HTTP Method:** *GET*
- **Request Parameters:** *title, city*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

  ```
  [
    {
      "averageRating": 4.2,
      "job": {
        "id": "234",
        "title": "Data Scientist",
        "companyName": "DataCorp",
  ```

```
      "description": "Exciting opportunity...",
      "location": "New York",
      "views": 78
    }
  }
]
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/search/jobs-in-best-companies

- **Path:** */jobs/search/jobs-in-best-companies*
- **HTTP Method:** *GET*
- **Request Parameters:** *title*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
[ { "id": "345", "title": "Analyst", "companyName": "TopCorp" } ]
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/search/best-paying-companies

- **Path:** */jobs/search/best-paying-companies*
- **HTTP Method:** *GET*
- **Request Parameters:** *title*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
[
  {
    "averageSalary": 120000,
    "company": {
      "id": "c2",
      "name": "FinTech Inc",
      "location": ["London"]
    }
  }
]
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/post/job-posting

- **Path:** */jobs/post/job-posting*
- **HTTP Method:** *POST*
- **Request Parameters:** *None*
- **Request Body Format (if any):** *JSON with job fields: id, title, companyName, description, location, views*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
{
  "message": "Job posting created successfully",
  "jobId": "abc123"
}
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/post/review-posting

- **Path:** */jobs/post/review-posting*
- **HTTP Method:** *POST*
- **Request Parameters:** *None*
- **Request Body Format (if any):** *JSON with review fields: id, companyId, rating, title, description*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
{
  "message": "Review submitted successfully",
  "reviewId": "rev789"
}
```

- **Error Response Format(s):** *400, 500*

Endpoint: /jobs/put/review-update

- **Path:** */jobs/put/review-update*
- **HTTP Method:** *PUT*
- **Request Parameters:** *None*
- **Request Body Format (if any):** *JSON with review_id and updated fields: rating, title, description*

- **Success Response Format:** *JSON, status code 200*
  Example:

```
{
  "message": "Review updated successfully",
  "reviewId": "rev789"
}
```

- **Error Response Format(s):** *400, 404, 500*

Endpoint: /jobs/delete/review-deletion

- **Path:** */jobs/delete/review-deletion*
- **HTTP Method:** *DELETE*
- **Request Parameters:** *review_id*
- **Request Body Format (if any):** *None*
- **Success Response Format:** *JSON, status code 200*
  Example:

```
{
  "message": "Review deleted successfully",
  "reviewId": "rev789"
}
```

- **Error Response Format(s):** *400, 404, 500*

gRPC Service Definitions (Internal)

Three internal gRPC services were defined to support data access and business logic across the system:

a. `DataAccessService` **(from** `data-access.proto`**):** Handles low-level data retrieval and persistence operations. Key RPC Methods:

- `GetJobPostingsWithTitle(JobPostingsRequestWithTitle)` → `JobPostingsResponse`
- `GetJobPostingsWithTitleAndCity(JobPostingsRequestWithTitleAndCity)` → `JobPostingsRespon`
- `GetJobReviewsWithTitleAndCity(JobReviewRequestWithTitleAndCity)` → `JobReviewsResponse`
- `CreateReview(CreateReviewRequest)` → `CreateReviewResponse`
- `UpdateJobReview(UpdateJobReviewRequest)` → `UpdateJobReviewResponse`
- `DeleteReview(DeleteReviewRequest)` → `DeleteReviewResponse`
- `GetRemoteJobs(RemoteJobSearchRequest)` → `RemoteJobSearchResponse`
- `PostJobInDB(PostJobRequest)` → `PostJobResponse`
- `GetBestPayingCompanies(BestPayingCompaniesRequest)` → `BestPayingCompaniesResponse`
- `GetCompaniesWithEmployees(CompaniesRequest)` → `CompaniesResponse`

b. `JobPostingService` **(from** `jobpostings.proto`**):** Handles business logic related to job posting management. Key RPC Methods:

- `AverageSalary(AverageSalaryRequest)` → `AverageSalaryResponse`
- `JobsWithRating(JobsWithRatingRequest)` → `JobsWithRatingResponse`
- `GetRemoteJobs(RemoteJobSearchRequest)` → `RemoteJobSearchResponse`
- `GetBestPayingCcompanies(BestPayingCompaniesRequest)` → `BestPayingCompaniesResponse`
- `AddJob(JobAddRequest)` → `JobAddResponse`
- `GetJobPostingsForLargestCompanies(JobPostingsForLargestCompaniesRequest)` → `JobPostingsForLargestCompaniesResponse`

c. `JobReviewService` **(from** `jobreviews.proto`**):** Responsible for operations on company and job reviews. Key RPC Methods:

- `CreateReview(CreateReviewRequest)` → `CreateReviewResponse`
- `UpdateJobReview(UpdateJobReviewRequest)` → `UpdateJobReviewResponse`
- `DeleteReview(DeleteReviewRequest)` → `DeleteReviewResponse`
- `GetBestCcompanies(BestCompaniesRequest)` → `BestCompaniesResponse`
- `CalculateRating(CalculateRatingRequest)` → `CalculateRatingResponse`
- `BestCity(BestCityRequest)` → `BestCityResponse`

3 Functional and Non-Functional Requirements

3.1 Functional Requirements (FRs)

- **FR1:** The system shall allow users to search for remote jobs filtered by city, keyword, or company.
- **FR2:** The system shall allow users to retrieve the average salary for a given job title.
- **FR3:** The system shall allow users to fetch a list of the best-rated companies based on reviews.
- **FR4:** The system shall allow users to retrieve a list of the best cities for job opportunities.
- **FR5:** The system shall allow users to search for jobs by title and city, returning results with average company ratings.
- **FR6:** The system shall allow users to retrieve job postings from the largest companies by employee count.
- **FR7:** The system shall allow users to retrieve companies offering the highest average salary for a given job title.
- **FR8:** The system shall allow users to post new job opportunities.

- **FR9:** The system shall allow users to submit new job reviews.
- **FR10:** The system shall allow users to update existing job reviews.
- **FR11:** The system shall allow users to delete existing job reviews.

Non-Functional Requirements (NFRs)

NFR-SEC-01 (Security – Secret Management): Sensitive configuration data (such as database credentials) must be stored securely and injected into application pods without being exposed in code or configuration files.

Implementation Discussion: Initially, Kubernetes Secrets were used for secret management. This was later enhanced by integrating **Google Cloud Secret Manager (GCSM)** with the **Secrets Store CSI Driver** and **Workload Identity**, ensuring secure and scalable secret injection at runtime.

NFR-SEC-02 (Security – Data Privacy): Implement measures to protect sensitive user or job data during processing and storage. Ensure encryption both in transit and at rest. Consider application-level encryption for specific sensitive fields.

Implementation Discussion:

- **Encryption at Rest:** Provided by default through Google Kubernetes Engine (GKE) for Persistent Disks (used by PostgreSQL) and Google Cloud Storage (used for CSV datasets, if applicable).
- **Encryption in Transit (External):** HTTPS on the Ingress is planned using Google-managed TLS certificates.
- **Encryption in Transit (Internal):** Achieved via **Anthos Service Mesh (ASM)** using **mTLS** between all microservices, including PostgreSQL, which was added to the mesh by enabling the sidecar and configuring strict PeerAuthentication policies.

4 Architecture

4.1 Application Architecture

Microservices:

- **api-interface:** Flask-based service that acts as the public API gateway. It handles incoming HTTP requests and orchestrates calls to backend gRPC services.
- **job-postings-service:** gRPC service responsible for handling logic related to job postings, such as searching and filtering.
- **job-reviews-service:** gRPC service responsible for handling logic related to job reviews.
- **data-access-service:** gRPC service that acts as a data abstraction layer, directly interacting with the PostgreSQL database.

Database:

- **PostgreSQL:** Centralized relational database that stores job postings, job reviews, and employee count data.

Communication:

- **gRPC:** Used for efficient inter-service communication between microservices.
- **HTTP:** Used for external API access via the Flask-based API interface.

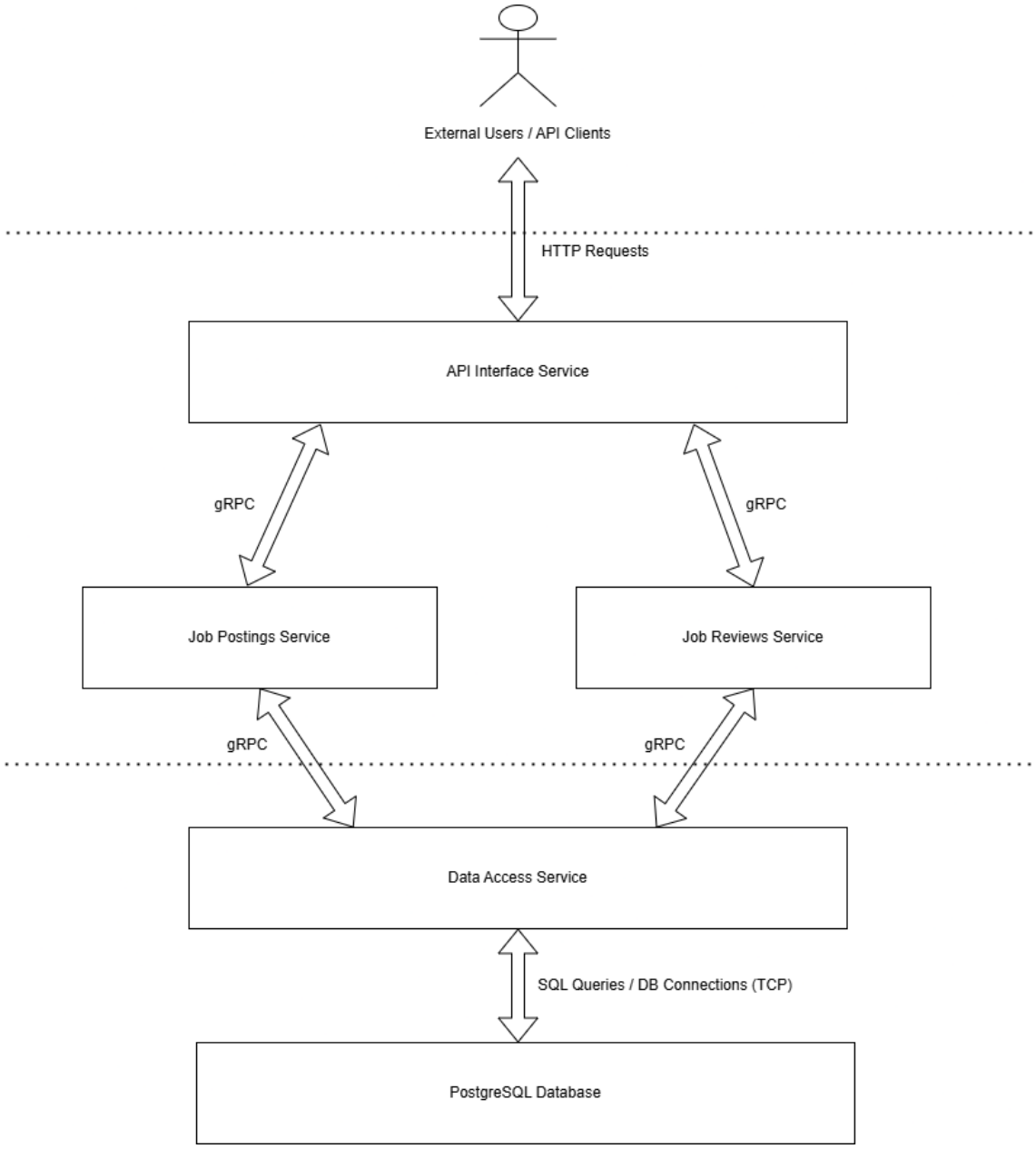


Figure 1: Application Architecture Diagram

4.2 Technical Architecture

- **Cloud Platform:** Google Cloud Platform (GCP).
- **Container Orchestration:** Google Kubernetes Engine (GKE) – Standard zonal cluster.
- **Service Mesh:** Anthos Service Mesh (ASM), with a Google-managed control plane, used for mTLS, traffic management (limited usage), and observability.

- **Database Hosting:** Self-managed PostgreSQL instance deployed as a `Deployment` on GKE with a `PersistentVolumeClaim`.
- **Container Registry:** Google Artifact Registry used to store and manage Docker images.
- **Secret Management:** Google Cloud Secret Manager (GCSM) integrated with GKE using the Secrets Store CSI Driver and Workload Identity for secure runtime secret injection.
- **Ingress:** GKE Ingress controller that provisions a Google Cloud HTTP(S) Load Balancer for external access.
- **CI/CD:** Google Cloud Build used for continuous integration and continuous deployment of microservices.

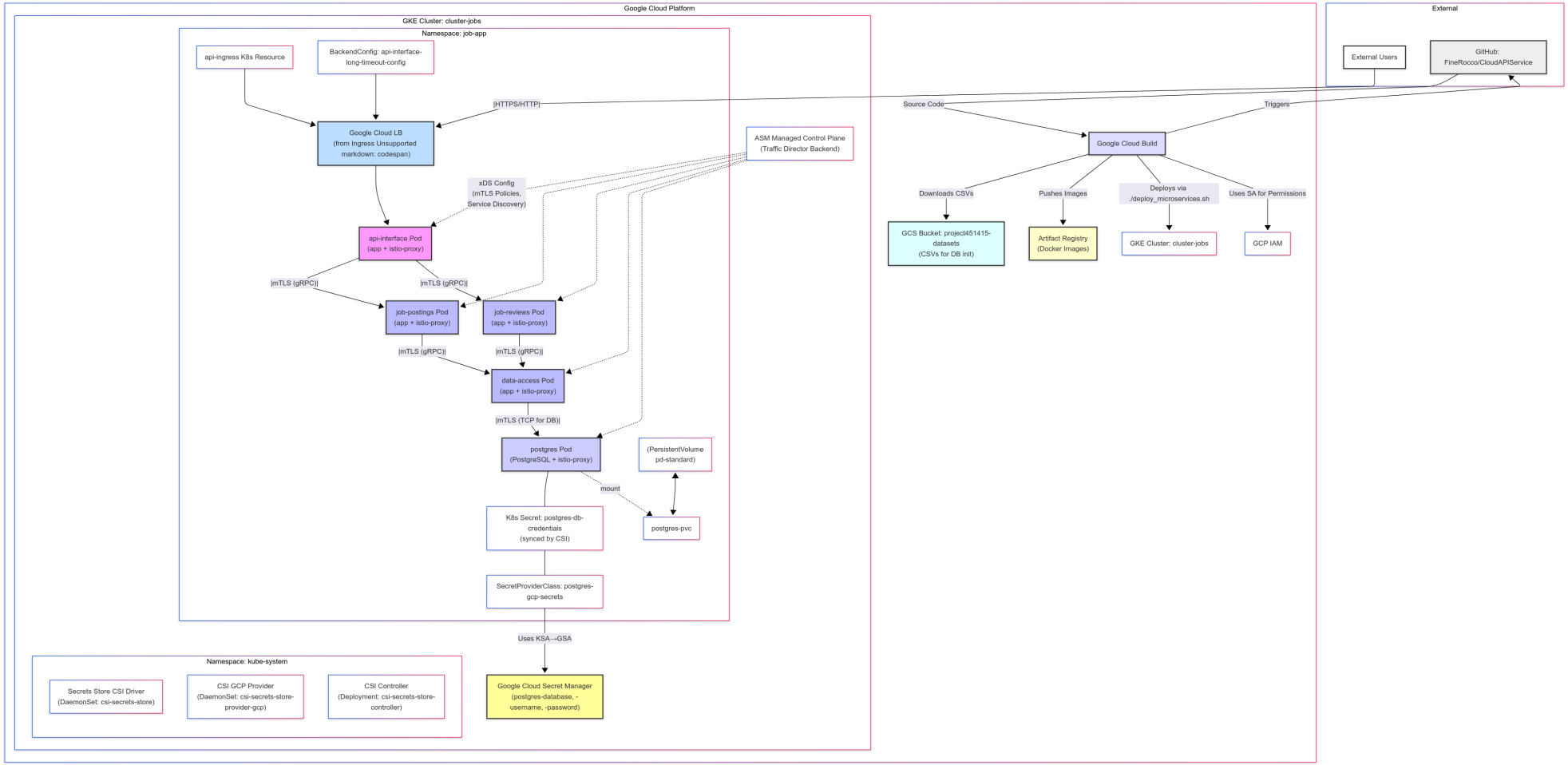


Figure 2: Technical Architecture Diagram

5 Deployment Scripts

5.1 `create_cluster.sh`

Purpose: Automates the creation of the GKE cluster (`cluster-jobs`).
Key Configurations:

- Zonal cluster in `europe-west1-b`.
- Initial node pool with a fixed number of `e2-standard-4` nodes (e.g., 2 nodes).
- `pd-standard` for boot disks (to manage SSD quota).
- Autoscaling disabled (due to GCE quotas).
- Workload Identity enabled.
- Necessary addons (e.g., `HttpLoadBalancing`, etc.).

5.2 `deploy_microservices.sh`

Purpose: Automates the deployment of all application components, service mesh configuration, and GCSM integration to an existing GKE cluster.
Key Stages:

1. **Authenticate to GKE:**
   - Ensures `gcloud` context is set to the target cluster.
2. **Install and Verify CSI Driver & GCP Provider:**
   - Installs Secrets Store CSI Driver and GCP Provider.
   - Configures RBAC for Kubernetes Secret sync.
   - Creates necessary `CSIDriver` object.
3. **Set Up Workload Identity:**
   - Creates GCP Service Account (`postgres-secrets-accessor`).
   - Creates Kubernetes Service Account (`postgres-ksa`).
   - Grants GSA permission to access GCSM secrets.
   - Binds GSA to KSA (Workload Identity annotation).
4. **Set Up Anthos Service Mesh (ASM):**
   - Enables required Google APIs.
   - Registers cluster to the fleet.
   - Configures managed ASM on the cluster.
   - Waits for ASM control plane readiness.
5. **Namespace & Istio Injection:**
   - Creates `job-app` namespace.
   - Labels namespace for automatic Istio sidecar injection.
6. **Apply SecretProviderClass:**

- Applies `SecretProviderClass` manifest for PostgreSQL credentials.

7. **Apply Core Kubernetes Manifests:**
   - `destination-rule.yaml` (for PostgreSQL mTLS client-side).
   - `peer-authentication.yaml` (for PostgreSQL mTLS server-side, if PostgreSQL is in the mesh).
   - `postgres-pvc.yaml`.
   - `postgres-service.yaml`.
   - `postgres-deployment.yaml` (using KSA, CSI volume, and image tag from CI/CD pipeline).
8. **Wait for CSI-Driven Secret Sync:**
   - Waits until the `postgres-db-credentials` Kubernetes Secret is synced by the CSI driver.
9. **Deploy Remaining Microservices:**
   - Applies Deployments and Services for:
     - `data-access`
     - `job-postings`
     - `job-reviews`
     - `api-interface`
   - Updates image tags for each microservice based on a script parameter (CI/CD).
10. **Apply Additional Configuration:**
    - Applies `BackendConfig` for `api-interface`.
    - Applies `Ingress` resource to expose relevant services externally.

6 CI/CD Pipeline

6.1 Platform

- Google Cloud Build

6.2 Source Repository

- GitHub: `https://github.com/FineRocco/CloudAPIService.git`

6.3 Trigger Configuration

- **On push to `main` branch:** Executes full build, image push, and deployment.
- **On Pull Requests to `main`:** Executes linting and unit tests (optionally builds images without pushing).

6.4 `cloudbuild.yaml` Key Stages

1. **(Optional/Example) Access GCSM Secrets**
   - Demonstrates build steps accessing Secret Manager secrets (e.g., build tools, API keys—not used for runtime app secrets).
2. **Code Quality Checks**
   - **Linting:** Runs `pylint` and/or `flake8` for each Python microservice.
   - **Dependencies:** Installs from `requirements.txt` for each service prior to linting.
3. **Unit Testing**
   - Placeholder in current `cloudbuild.yaml`—user to implement actual tests.
4. **Download CSV Datasets**
   - Uses `gsutil` to download:
     - `employee_counts.csv`
     - `job_postings.csv`
     - `job_reviews.csv`

     from GCS bucket (`gs://project451415-datasets/`) into the `./datasets/` directory in the build workspace.
5. **Build Docker Images**
   - Builds images for:
     - `postgres-with-data`
     - `api_interface`
     - `data_access`
     - `job_postings`
     - `job_reviews`
   - Uses correct build contexts:
     - `./datasets` for Postgres
     - `./microservices/<service-name>` for others (with `-f` flag pointing to Dockerfile)
   - Tags images with `$SHORT_SHA` (commit SHA) and `latest`.
6. **Push Docker Images**
   - Pushes tagged images to Google Artifact Registry:

     `europe-west1-docker.pkg.dev/projectcloud-451415/cloudapiservice`
7. **Deploy to GKE**
   - Retrieves GKE cluster credentials.
   - Executes `./deploy_microservices.sh`, passing `$SHORT_SHA` as the image tag argument.

8. **Service Account**

   - Cloud Build uses a designated service account (e.g., `data-access-sa@...` or default Cloud Build SA).
   - IAM roles granted include:
     - Artifact Registry Writer
     - Kubernetes Engine Admin
     - Service Account Creator/Admin
     - Secret Manager Admin/Accessor
     - Storage Object Viewer
     - Service Usage Admin
     - GKE Hub Admin

9. **Logging**

   - Configured to use `CLOUD_LOGGING_ONLY`.

7 Cost Analysis

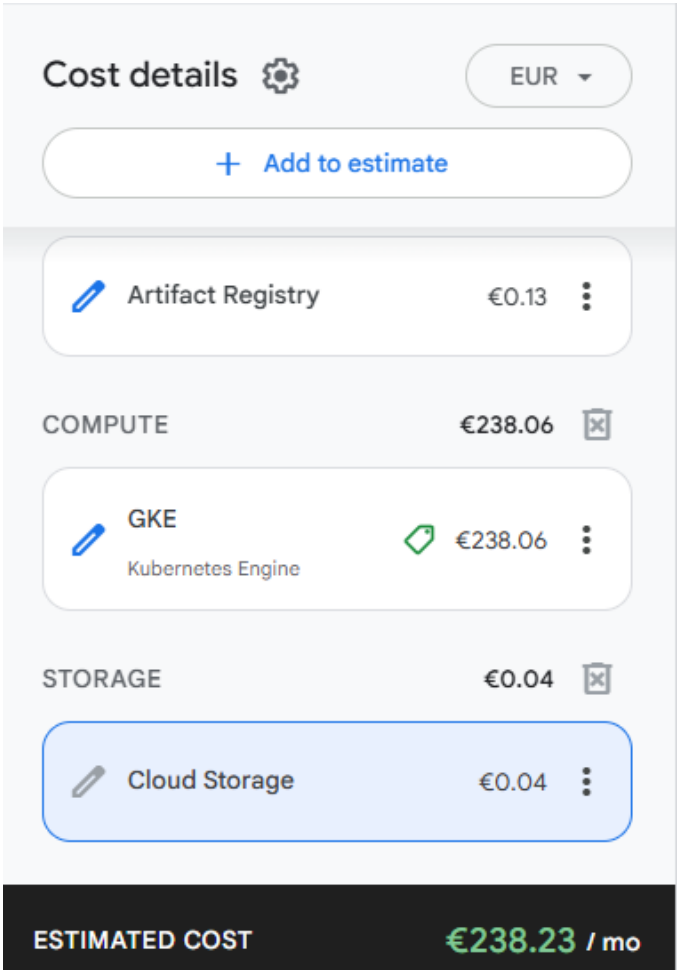


Figure 3: Google Costs

7.1 Components to Cost

- **GKE Cluster:**
  - **Control plane:** Standard zonal clusters usually have no separate fee, but verify for your configuration.
  - **Nodes:** 2 × `e2-standard-4` (4 vCPUs, 16 GB RAM each).
    Calculate based on hourly/monthly cost for this machine type in `europe-west1`.
- **Persistent Disks:**
  - **Node boot disks:** 2 × (e.g., 30 GB) `pd-standard`.
  - **PostgreSQL PVC:** 1 × 5 GB standard-RWO (maps to `pd-standard`).
- **Google Cloud Load Balancer:**
  - Cost for the HTTP(S) Load Balancer provisioned by GKE Ingress (includes forwarding rule, data processing).
- **Artifact Registry:**
  - Storage cost for Docker images.
  - Data transfer costs.
- **Cloud Build:**
  - Build minutes (consider the generous free tier first, then cost per minute beyond that).
- **Google Cloud Secret Manager:**
  - Cost per secret version stored.
  - Cost per access operation (CSI driver will access them). Note: Free tier usually covers typical small project usage.
- **Google Cloud Storage:**
  - Storage for CSV datasets (e.g., `gs://project451415-datasets/`).
  - Data transfer if accessed frequently.
- **Traffic Director (Anthos Service Mesh):**
  - ASM with a managed control plane on GKE is part of Anthos. If your project has an Anthos subscription or if usage exceeds any free tiers specifically for Traffic Director components used by ASM, there might be costs.
    (User to investigate current Anthos/ASM pricing for GKE).
- **IP Addresses:**
  - Cost for any static external IP addresses reserved.

7.2 FinOps Measures Implemented

- Selected cost-effective `e2-standard-4` machine types.
- Used `pd-standard` (standard persistent disk) for GKE node boot disks and PostgreSQL PVC instead of more expensive `pd-ssd`.
- Configured a fixed number of nodes (2) and disabled GKE cluster autoscaling to prevent unexpected costs and operate within strict GCE quotas.
- Optimized pod resource requests to the minimum viable levels to improve bin-packing and reduce resource waste on nodes.
- Implemented CI/CD to reduce manual effort and potential errors in deployment.

8 Test and Evaluation Techniques

To assess the robustness and performance of the `CloudAPIService` under both normal and high-load conditions, we carried out two main categories of tests: Stress Tests and Smoke Tests.

8.1 Stress Testing (Locust)

Tool Used: Locust.io
Test Configuration:

- **Concurrent Users:** 500
- **Endpoints Tested:** Creation and listing of communities, threads, and comments, as well as search and popular-content queries (e.g., `/communities`, `/threads`, `/comments`, `/search`, `/popular/...`).

Objectives:

- Evaluate performance metrics such as Requests Per Second (RPS) and response-time percentiles under load.

Key Results:

- **Functional Robustness:** No failed requests were recorded (0 failures), even under high concurrency.
- **Latency Degradation:** Response times degraded significantly; the 95th percentile reached 180s, and the median hit 60s at peak load.
- **Low Throughput:** RPS plateaued around 5 requests/s, indicating a performance bottleneck.
- **Bottleneck Endpoints:** The `/comments` endpoints were particularly slow, with average response times around 20 s.

8.2 Smoke Testing (Postman)

Smoke tests verified the correct behavior of the API for core operations:

- Successfully created and retrieved communities, threads, and comments.
- Identified a logic flaw: deleting a thread did not delete its associated comments, resulting in data inconsistency.

8.3 Summary and Implications

The API is functionally solid, as evidenced by the zero-failure rate. However, under stress conditions, the system exhibits severe performance degradation and scalability issues. Additionally, the data layer needs improvement to handle cascading deletions properly and ensure referential integrity.

9 Corrections Implemented After Feedback

This section details the significant evolutions and optimizations implemented in the project, encompassing both future database query improvements and the adjustments made to the infrastructure and application components.

9.1 Database Query Optimization for Performance and Cost Reduction

Three specific endpoints were exhibiting significant latency, taking too long to respond. Our initial hypothesis was that this inefficiency stemmed from complex or overly broad database queries.

To address this, we implemented a query optimization strategy focused on two main actions: limiting the scope of database queries and removing parameters that were not utilized in the business logic layer. The goal was to reduce the data processing overhead and minimize the computational load, thereby improving response times and reducing potential costs associated with prolonged resource usage.

However, despite these targeted optimizations, the performance of the three problematic endpoints did not improve as expected. The high latency issue persisted.

Further analysis suggests a potential correlation between the performance degradation and the recent implementation of security measures. It is our assessment that the root cause is most likely linked to the limited computational resources available on our student cloud account. The additional processing required by the security layer, combined with the existing query workload, appears to exceed the capacity of the allocated infrastructure.

Due to pressing time constraints, we were unable to conduct a more in-depth investigation or implement an alternative solution to this problem. Therefore, this remains a known issue for the project.

9.2 API Interface Enhancements and Routing Adjustments

This subsection describes the specific improvements made to the `api-interface` microservice, primarily focusing on API routing logic and parameter handling to align with RESTful best practices.

RESTful Routing for Resource Operations

- **DELETE `/reviews/{review_id}`:** The `render_delete_job_review` function's route was updated to capture `review_id` directly from the URL path. This ensures the resource identifier is a fundamental part of the URL, following standard RESTful conventions for deleting specific resources.
- **PUT `/reviews/{review_id}`:** Similarly, the `update_review` function's route was modified to extract `review_id` from the URL path. This aligns with RESTful best practices for updating specific resources, where the identifier is part of the URL and modifiable data is sent in the request body.

Consistent Parameter Handling

- **GET Endpoints:** Endpoints such as `/jobs/search/average-salary` and `/jobs/search/remote` continue to use query parameters (`request.args.get()`) as the appropriate method

for filtering and searching.

- **POST Endpoints:** Endpoints such as `/jobs/post/job-posting` and `/jobs/post/review-posting` continue to accept data via the JSON request body (`request.get_json()`) for creating new resources. Upon successful creation, the response now includes the unique ID of the newly created object in the success message, allowing for immediate reference or subsequent operations.

These changes collectively enhance the API's design, making it more semantically correct and consistent with widely accepted RESTful principles for resource manipulation.

10 Conclusion

Overall, our `CloudAPIService` proves to be an effective solution for analyzing labor market data. The platform enables optimized job search, salary trend analysis, and comparisons between companies and cities based on relevant criteria such as organizational culture and work–life balance. Tailored to the needs of candidates, the system serves as an essential tool for making informed decisions in job search and selection, standing out as a highly valuable resource for labor market analysis.

10.1 Key Achievements

- Implemented a RESTful API backed by gRPC-connected microservices handling business logic and data access.
- Leveraged cloud solutions for deployment and management:
  - Google Kubernetes Engine (GKE) for containerization and deployment.
  - Anthos Service Mesh (ASM) for traffic management and enhanced security.
  - Google Cloud Secret Manager (CGSM) for secure storage of sensitive data.
- Established a CI/CD pipeline with Google Cloud Build, enabling faster and more reliable deployments.

10.2 Obstacles Faced

- Limited student credits on GCP:
  - We found the 50$ student credit for GCP too small. A simple "mistake" in the early stages of the project would easily spend a significative percentage of those 50$. We spent this credit very early on and had to resort to the free 300$ trial that Google offers. This 300$ credit trial is not very convenient as it required us to add a personal credit card to the billing account.
  - The quota limits imposed on our student plan significantly hindered progress on security improvements. We couldn't deploy clusters with the necessary resources because we hit these limits. Unfortunately, as a student account, we weren't able to request a higher quota to address this issue.
- Operational challenges with GKE:
  - Arbitrary problems in the GKE pods which prompted restarts in the cluster that could take 30mins or more.
  - The Kubernetes cluster sometimes would take too long to start, causing delays in the development of the project.

10.3 Final Considerations

By making this project we gained valuable experience in architecting, deploying and operating a cloud-native application. We also learned important lessons of scalability, security and cost control which we think we'll be extremely important moving forward.